Los Aromos - Documento de diseño de etapa 1

Propuesta A: Gestión de un barrio cerrado de 150 lotes.

Integrantes: Perez Calle Maribel (1172744), Juan Ignacio Vara (1216613), Luka Subotovsky (1207995).

Compilación preparada el 06/11/2026 a partir del repositorio LukaSubotovsky/Paradigma-orientado-a-objetos, versión revisada 223c6cfd0f637ee17ef82be594fd71e85b930ea4.

Contenido

1. Informe existente: actores, requisitos, reglas, supuestos y justificación SOLID/GRASP.

2. Complemento: descripciones de casos de uso y trazabilidad.

3. Nota comparativa del diseño inicial y final.

4. Diagramas existentes: casos de uso, clases inicial, secuencias y clases final.

Estado del documento

Versión integrada para revisión grupal. Se conservan las imágenes y el informe del repositorio. La compilación agrega las secciones faltantes; no modifica los errores de los diagramas originales. Las fuentes editables reconstruidas se entregan aparte y no son los archivos originales de los autores.

Propuesta: **Gestión de un barrio cerrado**

Integrantes:
Perez Calle Maribel 1172744
Juan Ignacio Vara 1216613
Luka Subotovsky 1207995

Contexto de la propuesta: un barrio cerrado con 150 lotes necesita un sistema para ordenar la relación entre la administración y los vecinos, que hoy se resuelve con planillas y mensajes sueltos.
Etapa 1: Modelado y aplicación de patrones
1. Análisis del enunciado:

ACTORES:

- Administrador: es el encargado de la gestión del barrio.
- Titular del Lote: presenta reclamos, paga expensas, autoriza visitas y proveedores, y recibe las notificaciones formales de morosidad.
- Personal autorizado por el titular: presenta reclamos en nombre del titular.
- Personal guardia de Seguridad: personal encargado de la portería, ronda o monitoreo.
- Personal empleado de mantenimiento: personal técnico con una especialidad.
- Residentes(inquilino/familiares): Vive en el lote y puede autorizar visitas
- Visitante/Proveedor: no usa el sistema, pero sus datos se registran al ingresar y egresar.

REQUERIMIENTOS FUNCIONALES:
Reclamos y solicitudes

- **RF1:** Registrar un reclamo o solicitud con tipo, descripción, lote y persona que lo presenta. El sistema asigna número, fecha, prioridad y estado inicial.
- **RF2:** Asignar un reclamo a un empleado.
- **RF3:** Avanzar el estado de un reclamo y registrar fecha, nuevo estado y observación en cada cambio.
- **RF4:** Reabrir un reclamo resuelto dentro del plazo permitido.
- **RF5:** Agregar comentarios del vecino y del personal a un reclamo.
- **RF6:** Consultar el historial completo de estados de un reclamo.
- **RF7:** Escalar al administrador los reclamos de prioridad alta no atendidos a tiempo.

Expensas y pagos

- **RF8:** Cargar el presupuesto mensual y emitir una expensa por lote, calculada con el coeficiente de participación.
- **RF9:** Registrar pagos (fecha, importe, medio de pago) y aplicarlos a una o más expensas.
- **RF10:** Calcular el interés por mora sobre el saldo impago.
- **RF11:** Imputar los pagos parciales según el orden definido.
- **RF12:** Marcar un lote como moroso, generar la notificación formal al titular y quitar la condición al regularizar.
- **RF13:** Registrar asientos de ajuste para corregir pagos erróneos.

Luminarias y cámaras

- **RF14:** Dar de alta equipos (cámaras y luminarias) con sus datos propios.
- **RF15:** Registrar la falla de un equipo y cambiar su estado.
- **RF16:** Generar una orden de mantenimiento con prioridad automática y asignarla a un empleado.
- **RF17:** Cerrar una orden registrando fecha, técnico y descripción de la reparación.
- **RF18:** Consultar el estado actual y el historial de órdenes de cada equipo.
- **RF19:** Informar al administrador cuando una cámara supera el tiempo máximo sin funcionar.

Visitas y proveedores

- **RF20:** Registrar la autorización de una visita (visitante, lote, fecha y franja horaria), puntual o recurrente.
- **RF21:** Consultar una autorización y habilitar o denegar el ingreso.
- **RF22:** Registrar el ingreso y el egreso de cada persona.

Reportes y datos maestros

- **RF23:** Emitir los reportes de reclamos pendientes, morosidad por lote y equipos fuera de servicio.
- **RF24.** Gestionar lotes, titulares, residentes, personas autorizadas y personal.

REGLAS DE NEGOCIOS:

Reclamos y solicitudes

- **RN1:** Tipos posibles: seguridad, mantenimiento, convivencia y administrativo.
- **RN2:** La prioridad se deriva del tipo: seguridad es alta, mantenimiento es media, convivencia y administrativo son bajas.
- **RN3:** Los estados son abierto, asignado, en curso, resuelto y cerrado, y solo se puede avanzar al siguiente.
- **RN4:** El vecino puede reabrir un reclamo resuelto dentro de 5 días. Pasado ese plazo sin respuesta, el reclamo se cierra.
- **RN5:** Cada reclamo tiene un único responsable asignado.
- **RN6:** Ningún empleado puede tener más de 5 reclamos en curso a la vez.
- **RN7:** Un reclamo de prioridad alta no atendido en 24 horas se escala al administrador.
- **RN8:** Solo el titular del lote o una persona autorizada por él puede presentar reclamos.

Expensas y pagos

- **RN9:** Expensa del lote = presupuesto del mes × coeficiente de participación (según superficie).
- **RN10:** El vencimiento es el día 10 de cada mes. Desde esa fecha se cobra un interés punitorio del 3 % mensual sobre el saldo impago.
- **RN11:** Un pago parcial se imputa primero a intereses y después a capital, empezando por el período más antiguo.

- **RN12:** Un lote con más de 3 períodos impagos es moroso y se notifica formalmente al titular. La condición se quita al regularizar.
- **RN13:** Un pago registrado no se elimina. Los errores se corrigen con un asiento de ajuste.

Luminarias y cámaras

- **RN14:** Estados de un equipo: operativo, con falla, en reparación o fuera de servicio.
- **RN15:** Una falla en cámara genera una orden de prioridad alta. Una falla en luminaria genera una de prioridad normal, que pasa a alta si fallan más de 3 luminarias del mismo sector.
- **RN16:** Si una cámara está sin funcionar más de 48 horas, se informa al administrador.
- **RN17:** Una orden no se puede cerrar sin fecha, técnico y descripción de la reparación.

Visitas

- **RN18:** Toda visita requiere autorización previa del vecino, con fecha y franja horaria de vigencia. Sin autorización, no se habilita el ingreso.
- **RN19:** Los proveedores habituales pueden tener una autorización recurrente de hasta 30 días.
- **RN20:** Cada ingreso se registra con su egreso. Una persona no puede registrar un segundo ingreso sin haber salido.

Estructurales

- **RN21:** El barrio tiene 150 lotes. Cada lote tiene un único titular, y una persona puede ser titular de varios lotes.
- **RN22:** Un lote puede tener residentes adicionales (familiares o inquilinos).
- **RN23:** Un equipo puede tener muchas órdenes de mantenimiento a lo largo de su vida.

SUPUESTOS:

Nº	**Ambigüedad detectada**	**Supuesto adoptado**	**Justificación**
S1	El enunciado habla de “reclamos y solicitudes” pero solo define reclamos.	Una solicitud es un reclamo más y sigue el mismo ciclo de vida, los mismos tipos y las mismas reglas.	Evita duplicar el modelo.

S2	Al reabrir un reclamo resuelto, no se dice a qué estado vuelve, aunque “solo se avanza”.	La reapertura es la única excepción a RN3: vuelve a “asignado” con el mismo responsable, y queda registrada en el historial.	Conserva la trazabilidad y al responsable.
S3	“Sin respuesta” en 5 días no indica quién debe responder.	Los 5 días son corridos desde que pasa a “resuelto”. Si el vecino no lo reabre, se cierra automáticamente.	Regla simple y verificable.
S4	“Empleado” en la asignación puede ser mantenimiento o guardia.	Se puede asignar a cualquier miembro del personal. Se recomienda guardia para seguridad y mantenimiento para mantenimiento. Convivencia y administrativo los atiende cualquiera.	El enunciado no restringe la asignación.
S5	El límite de 5 reclamos “en curso” no indica cuándo se valida.	Se valida al pasar un reclamo a “en curso”. Un empleado puede tener más reclamos asignados, pero no más de 5 en curso.	Respeta literalmente la regla.
S6	“No atendido en 24 horas” no define qué es atender.	Se cuentan 24 horas corridas desde la apertura. Un reclamo se considera atendido cuando pasa a “en curso”.	Es el primer estado que implica trabajo real.
S7	“Persona autorizada” no tiene datos definidos.	Se registra con los mismos datos personales que un residente y la autoriza el titular.	Reutiliza la entidad persona.
S8	El coeficiente de participación no tiene fórmula.	Coeficiente = superficie del lote ÷ suma de superficies de los 150 lotes. El presupuesto lo carga el administrador cada mes.	Es la definición habitual y los coeficientes suman 1.

S9	Se cobra “3 % mensual” sin aclarar el cálculo.	Interés simple, sin capitalizar, prorrateado por día (3 % ÷ 30) sobre el capital impago desde el día 11. Un pago hecho hasta el día 10 no genera interés.	Es un cálculo claro y fácil de auditar.
S10	Los estados de la expensa no se listan.	Pendiente, parcialmente pagada, pagada y vencida.	Cubre las situaciones que se deducen de las reglas.
S11	“Más de tres períodos impagos” es ambiguo.	Se considera moroso con 4 o más expensas vencidas con saldo pendiente. La notificación se genera una sola vez al alcanzar la condición.	“Más de tres” implica cuatro o más.
S12	Un pago puede cubrir una o varias expensas.	Un pago se aplica a varias expensas según la imputación de RN11. Cada aplicación queda registrada.	Permite reconstruir cómo se cancela cada deuda.
S13	Se menciona “orden de prioridad normal”, que no figura en la prioridad de reclamos (media).	Las órdenes usan dos niveles: normal y alta.	Respeta el texto del enunciado.
S14	“Más de 3 luminarias del mismo sector” no indica si cuentan simultáneas o acumuladas.	Cuentan las luminarias del sector que están con falla o en reparación al mismo tiempo. Al superar 3, las órdenes abiertas de ese sector pasan a alta.	Refleja el impacto actual en el sector.

S15	“Fuera de servicio” no se diferencia de “con falla”.	“Con falla” se asigna al registrar la falla. “En reparación” cuando la orden pasa a ejecución. “Operativo” al cerrar la orden. “Fuera de servicio” lo decide el administrador cuando el equipo no se puede reparar en forma inmediata o se retira.	Da sentido a los cuatro estados.
S16	Las 48 horas de la cámara no indican desde cuándo se cuentan.	Se cuentan desde el registro de la falla, y se avisa una sola vez.	Evita avisos repetidos.
S17	“El vecino” que autoriza visitas no se define.	Pueden autorizar el titular y los residentes registrados del lote.	Los residentes también reciben visitas.
S18	No se aclara cuántos ingresos permite una autorización.	Una autorización puntual habilita un solo ingreso en su franja. Una recurrente habilita un ingreso por día, en la franja diaria indicada, hasta 30 días.	Es coherente con RN20.
S19	No se define qué pasa con un egreso no registrado.	El guardia puede registrar un egreso manual con observación.	Evita que una persona quede bloqueada.
S20	El visitante se identifica por nombre y documento.	El documento es el identificador único del visitante.	Permite verificar RN20.
S21	Los reportes no tienen filtros definidos.	Los reportes se filtran por fecha, y por sector o tipo según corresponda. Morosidad incluye deuda de capital e intereses.	Los vuelve útiles sin cambiar el alcance.

S22	No se menciona el control de accesos al sistema.	Cada actor ingresa con usuario y rol, y solo ve las funciones que le corresponden.	Es necesario para aplicar RN8, RN18 y RN20.
S23	El cambio de titular de un lote no se contempla.	Fuera de alcance: se considera un titular por lote durante todo el trabajo.	Mantiene acotado el modelo.

5. Aplicación de patrones: 3solid y 4grasp
SOLID:

S: Responsabilidad única

- Problema detectado:si Reclamo guardaba sus datos y además validaba transiciones, escalaba por tiempo y controlaba el límite de 5 reclamos en curso por empleado, cambiaba por varios motivos distintos. Lo mismo pasaba con el cálculo de intereses dentro de Expensa.
- Decisión: Reclamo conserva solo sus datos y su historial. Las reglas de asignación, transición, reapertura, escalamiento y límite de carga pasan a GestorReclamos. El cálculo de mora pasa a CalculadorIntereses. Los roles de una persona en un lote van en VinculoLote, no en Persona.
- Dónde se ve: Reclamo, GestorReclamos, Expensa, CalculadorIntereses, Persona, Lote, VinculoLote.
- Ventaja: un cambio en una regla (la tasa de mora o el límite de 5) se hace en una sola clase y no puede romper el resto.

O: Abierto/cerrado

- Problema detectado: la prioridad de la orden de mantenimiento depende del tipo de equipo (cámara siempre alta, luminaria normal o alta). Resolverlo con if tipo == camara obligaría a modificar código existente cada vez que aparezca un tipo nuevo.
- Decisión: Equipo declara un método abstracto prioridadOrden() y cada subclase lo implementa. Del mismo modo, cada constante de TipoReclamo guarda su PrioridadReclamo, y los roles de lote son valores de RolLote.
- Dónde se ve: Equipo, Camara, Luminaria, TipoReclamo, PrioridadReclamo, RolLote.
- Ventaja: agregar un tipo de equipo, un tipo de reclamo o un rol nuevo se hace sumando una subclase o una constante, sin tocar las clases que ya funcionan.

L: Sustitución de Liskov

- Problema detectado: al heredar Administrador de Personal, un administrador podría asignarse como responsable de reclamos u órdenes, cosa que el enunciado no

prevé. Algo parecido pasaba con Titular y Residente como subclases de Persona: una persona con dos roles no podía representarse.
- Decisión: Administrador hereda de Persona y no de Personal. Titular, residente y autorizado se modelan como roles (RolLote), no como subclases. Las únicas jerarquías que quedan son las que sí cumplen “es un”: guardia y empleado de mantenimiento son Personal, cámara y luminaria son Equipo.
- Dónde se ve: Persona, Administrador, Personal, GuardiaSeguridad, EmpleadoMantenimiento, Equipo, Camara, Luminaria.
- Ventaja: cualquier clase que reciba un Personal o un Equipo puede usar cualquier subclase sin verificar de cuál se trata y sin que cambie su comportamiento esperado.

GRASP

Experto en información

- Problema detectado: al emitir expensas o armar reportes, los cálculos podían terminar en el controlador o en una clase de reportes que “sabe de todo”, copiando datos de otras clases.
- Decisión: cada cálculo vive en la clase que tiene los datos. Expensa calcula su saldo, Lote calcula su deuda y decide quién puede presentar reclamos, Barrio calcula los coeficientes y reúne reclamos y equipos, Reclamo dice si está pendiente, Equipo dice si está fuera de servicio, RegistroAcceso dice si la persona está adentro.
- Dónde se ve: Expensa.calcularSaldo(), Lote.calcularDeudaCapital(), Lote.puedePresentarReclamo(), Barrio.calcularCoeficientes(), Reclamo.estaPendiente(), Equipo.estaFueraDeServicio(), RegistroAcceso.estaAdentro().
- Ventaja: la lógica queda junto a los datos que usa, los atributos pueden ser privados y cada clase se puede probar sola.

Creador

- Problema detectado: había que decidir quién instancia los objetos dependientes (historial, comentarios, vínculos, aplicaciones de pago) para no crearlos desde cualquier lugar.
- Decisión: crea cada objeto la clase que lo contiene y lo usa más de cerca, lo que coincide con las composiciones. Barrio crea sus Lote, Lote crea sus VinculoLote, Reclamo crea CambioEstado y Comentario, Pago crea AplicacionPago y AsientoAjuste, Autorizacion crea RegistroAcceso.
- Dónde se ve: todas las relaciones de composición del diagrama.
- Ventaja: el ciclo de vida de la parte queda atado al del todo (si no existe el reclamo, no existe su historial) y ninguna otra clase necesita conocer cómo se construyen.

Controlador

- Problema detectado: sin un punto de entrada definido, la interfaz podría llamar directamente a las clases del dominio y mezclar presentación con reglas de negocio.

- Decisión: un controlador por grupo de casos de uso, que recibe la operación y la delega sin contener reglas: ControladorReclamos, ControladorExpensas, ControladorEquipos, ControladorVisitas y ControladorReportes.
- Dónde se ve: las cinco clases de control y sus dependencias con el dominio.
- Ventaja: la interfaz (pantalla, web, móvil) se puede cambiar sin tocar el dominio, y las reglas no se duplican entre pantallas.

Alta cohesión

- Problema detectado: un controlador que además filtraba, sumaba deudas y decidía reglas habría mezclado coordinación, cálculo y negocio.
- Decisión: cada clase se ocupa de un solo tema. Los controladores solo coordinan, Lote y Expensa calculan, Configuracion solo guarda parámetros, FilaMorosidad solo transporta datos de un reporte.
- Dónde se ve: ControladorReportes frente a Lote, Barrio, Reclamo y Equipo.
- Ventaja: las clases son pequeñas, se entienden por su nombre y es más fácil ubicar dónde corregir un error.

Bajo acoplamiento

- Problema detectado: si Reclamo consultara directamente cómo se guardan titulares y autorizados, o si los controladores se llamaran entre sí, un cambio en una clase se propagaría a muchas otras.
- Decisión: Reclamo solo le pregunta a Lote si una persona puede reclamar. Los controladores no se conocen entre sí. Las enumeraciones se usan solo como tipo de atributo. Los reportes llegan a los datos a través de Barrio.
- Dónde se ve: Lote.puedePresentarReclamo(), los cinco controladores sin relaciones entre ellos, ControladorReportes → Barrio.
- Ventaja: se puede modificar la forma de guardar los vínculos de un lote sin tocar Reclamo, y los módulos (reclamos, expensas, equipos, visitas) se pueden desarrollar de forma independiente.

Polimorfismo

- Problema detectado: el comportamiento varía según el tipo de equipo y según el tipo de personal, y resolverlo con condicionales dispersos hace el código frágil.
- Decisión: el comportamiento que varía se declara en la clase abstracta y lo resuelve cada subclase. Equipo.prioridadOrden() lo implementan Camara y Luminaria. Los reclamos y las órdenes se asignan a un Personal, sin importar si es guardia o empleado.
- Dónde se ve: Equipo, Camara, Luminaria, Personal, Reclamo, OrdenMantenimiento.
- Ventaja: quien usa un equipo o un miembro del personal lo trata de forma uniforme, y cada tipo decide cómo responde.

Fabricación pura

- Problema detectado: cargar a Expensa con el cálculo de interés, o a Reclamo con las reglas de gestión, les agregaba responsabilidades que no son conceptos del barrio y bajaba su cohesión.
- Decisión: se crearon clases artificiales, que no existen en el enunciado, para sostener esa lógica: GestorReclamos, CalculadorIntereses, Configuracion y FilaMorosidad.
- Dónde se ve: esas cuatro clases y sus dependencias con Reclamo, Expensa y los controladores.
- Ventaja: las clases del dominio quedan simples y fieles al enunciado, y la lógica auxiliar se puede reutilizar o reemplazar.

Variaciones protegidas

- Problema detectado: el enunciado dice que los valores numéricos son de referencia y el grupo puede ajustarlos (3 % de mora, día 10, 5 días de reapertura, 24 horas de escalamiento, máximo 5 en curso). Si estuvieran escritos dentro de las reglas, cada ajuste obligaría a modificar varias clases.
- Decisión: los valores se concentran en Configuracion, y GestorReclamos y CalculadorIntereses los leen de ahí. La correspondencia tipo-prioridad queda en las constantes de TipoReclamo.
- Dónde se ve: Configuracion, GestorReclamos, CalculadorIntereses, TipoReclamo.
- Ventaja: cambiar un plazo o una tasa se hace en un solo lugar, sin tocar la lógica que lo usa.

Descripciones de casos de uso - Los Aromos

Propuesta A. Integrantes: Perez Calle Maribel (1172744), Juan Ignacio Vara (1216613), Luka Subotovsky (1207995).

Complemento del diseño del repositorio revisado. Los casos desglosan los RF1-RF24; varios subprocesos se agrupan en un mismo caso. Los casos automáticos describen comportamiento del sistema, sin convertir al sistema en actor externo. Los roles de edición y algunas validaciones no definidos en el enunciado se indican como supuestos complementarios y deben ser aceptados por el grupo.

CU01. Registrar reclamo

Actor: Titular o persona autorizada.

Precondiciones: Lote y persona registrados; vínculo habilitado.

Flujo principal:

1. Seleccionar lote.

2. ingresar tipo y descripción.

3. validar autorización.

4. asignar número, fecha, prioridad y estado abierto.

5. guardar reclamo e historial inicial.

Alternativas / errores: Sin autorización: rechazar. Datos incompletos: solicitar completar.

Postcondición: Reclamo abierto y vinculado al lote y al solicitante.

CU02. Asignar reclamo

Actor: Administrador.

Precondiciones: Reclamo abierto y miembro del personal registrado.

Flujo principal:

1. Seleccionar reclamo y responsable.

2. registrar asignación.

3. avanzar a asignado.

4. guardar fecha y observación.

Alternativas / errores: Responsable inexistente: rechazar. Estado incompatible: no modificar. El máximo de cinco se controla al pasar a en curso, según S5.

Postcondición: Un único responsable asignado y cambio registrado.

CU03. Consultar y seguir reclamo

Actor: Vecino, administrador o personal responsable.

Precondiciones: Reclamo existente y acceso permitido.

Flujo principal:

1. Buscar reclamo.

2. mostrar estado, prioridad, responsable, historial y comentarios.

Alternativas / errores: Reclamo inexistente o acceso no permitido: informar sin mostrar datos.

Postcondición: Información consultada sin modificar el reclamo.

CU04. Avanzar estado de reclamo

Actor: Administrador o personal responsable.

Precondiciones: Reclamo asignado o en curso.

Flujo principal:

1. Seleccionar reclamo.

2. solicitar siguiente estado.

3. validar transición.

4. si pasa a en curso, contar reclamos del responsable.

5. registrar estado, fecha y observación.

Alternativas / errores: Salto de estado: rechazar. Cinco reclamos ya en curso: impedir un sexto.

Postcondición: Estado actualizado y trazabilidad conservada.

CU05. Reabrir reclamo

Actor: Titular o persona autorizada.

Precondiciones: Reclamo resuelto del lote; dentro de cinco días corridos.

Flujo principal:

1. Solicitar reapertura.

2. validar permiso y plazo.

3. volver a asignado con el mismo responsable.

4. registrar observación e historial.

Alternativas / errores: Plazo vencido o estado distinto de resuelto: rechazar.

Postcondición: Reclamo asignado nuevamente, conservando el historial.

CU06. Agregar comentario

Actor: Vecino habilitado o personal.

Precondiciones: Reclamo existente; usuario autorizado.

Flujo principal:

1. Seleccionar reclamo.

2. ingresar texto.

3. registrar comentario con autor y fecha.

Alternativas / errores: Texto vacío o falta de permiso: no guardar.

Postcondición: Comentario agregado sin alterar comentarios previos.

CU07. Cerrar reclamo

Actor: Administrador; sistema en cierre automático.

Precondiciones: Reclamo resuelto.

Flujo principal:

1. Verificar estado y plazo.

2. registrar cierre e historial. Para cierre automático, comprobar cinco días corridos sin reapertura.

Alternativas / errores: Reclamo reabierto o aún no resuelto: no cerrar.

Postcondición: Reclamo cerrado y evento de cierre registrado.

CU08. Escalar reclamos urgentes

Actor: Sistema, con destinatario administrador.

Precondiciones: Reclamos de seguridad abiertos o asignados.

Flujo principal:

1. Revisar fechas.

2. detectar prioridad alta sin pasar a en curso en 24 horas.

3. informar al administrador y dejar constancia.

Alternativas / errores: Si ya fue atendido, no escalar. Revisión repetida: evitar avisos duplicados (supuesto operativo complementario).

Postcondición: Administrador informado de los reclamos urgentes pendientes.

CU09. Emitir expensas

Actor: Administrador.

Precondiciones: Lotes y superficies cargados; presupuesto del período disponible.

Flujo principal:

1. Ingresar período y presupuesto.

2. calcular coeficientes por superficie.

3. emitir una expensa por lote.

4. fijar vencimiento el día 10.

5. guardar resultados.

Alternativas / errores: Superficie total inválida: impedir emisión. Período ya emitido: evitar duplicación (supuesto complementario).

Postcondición: Expensas del período registradas.

CU10. Registrar pago e imputarlo

Actor: Administrador.

Precondiciones: Lote con expensas; importe positivo y medio válido.

Flujo principal:

1. Ingresar fecha, importe y medio.

2. calcular intereses.

3. ordenar períodos del más antiguo al más reciente.

4. imputar primero intereses y luego capital.

5. guardar pago y aplicaciones.

6. actualizar saldos y morosidad.

Alternativas / errores: Importe inválido: rechazar. Pago mayor que la deuda: solicitar un importe igual o menor (supuesto complementario, sin saldo a favor).

Postcondición: Pago y aplicaciones conservados; saldos actualizados.

CU11. Calcular intereses por mora

Actor: Administrador o proceso de pagos/reportes.

Precondiciones: Expensa con vencimiento y saldo de capital conocidos.

Flujo principal:

1. Tomar fecha de cálculo.

2. contar días desde el día 11.

3. aplicar interés simple diario de 3 % / 30 al capital pendiente, respetando fechas de pagos.

Alternativas / errores: Hasta el día 10 o sin capital pendiente: interés cero.

Postcondición: Interés disponible para imputación o consulta.

CU12. Verificar y notificar morosidad

Actor: Administrador o sistema.

Precondiciones: Expensas y aplicaciones de pago actualizadas.

Flujo principal:

1. Contar períodos vencidos con saldo.

2. marcar moroso con cuatro o más.

3. emitir notificación al titular al alcanzar la condición.

4. quitar marca cuando deje de cumplirla.

Alternativas / errores: Condición ya notificada: no duplicar aviso. Menos de cuatro períodos: no marcar moroso.

Postcondición: Condición y notificaciones consistentes con la deuda.

CU13. Registrar ajuste de pago

Actor: Administrador.

Precondiciones: Pago existente con un error identificado.

Flujo principal:

1. Seleccionar pago.

2. ingresar motivo e importe de corrección.

3. registrar asiento relacionado.

4. recalcular aplicaciones, saldos y morosidad afectados.

Alternativas / errores: Motivo vacío o ajuste incompatible: rechazar (validación complementaria). No eliminar el pago original.

Postcondición: Ajuste trazable y pago original conservado.

CU14. Registrar equipo

Actor: Administrador.

Precondiciones: Código de equipo disponible.

Flujo principal:

1. Elegir cámara o luminaria.

2. ingresar código, sector, fecha y datos específicos.

3. guardar equipo y estado inicial.

Alternativas / errores: Código duplicado o datos inválidos: rechazar. Estado inicial operativo salvo indicación expresa (supuesto complementario).

Postcondición: Equipo registrado con datos de su tipo.

CU15. Registrar falla de equipo

Actor: Vecino o guardia de seguridad.

Precondiciones: Equipo existente.

Flujo principal:

1. Seleccionar equipo.

2. ingresar descripción.

3. registrar fecha y reportante.

4. pasar equipo a con falla.

5. dejar falla disponible para mantenimiento.

Alternativas / errores: Equipo inexistente: rechazar. Falla repetida: informar el registro vigente antes de duplicar (supuesto complementario).

Postcondición: Falla registrada y estado actualizado.

CU16. Generar orden de mantenimiento

Actor: Administrador.

Precondiciones: Falla registrada y personal disponible.

Flujo principal:

1. Seleccionar equipo y responsable.

2. crear orden con apertura y descripción.

3. cámara: prioridad alta.

4. luminaria: normal, alta si hay más de tres falladas o en reparación en el sector.

5. actualizar órdenes abiertas del sector.

Alternativas / errores: Datos incompletos: no generar. Al iniciar la reparación, pasar equipo a en reparación.

Postcondición: Orden asignada con prioridad correspondiente.

CU17. Cerrar orden de mantenimiento

Actor: Empleado de mantenimiento.

Precondiciones: Orden abierta con reparación terminada.

Flujo principal:

1. Seleccionar orden.

2. registrar fecha, técnico y descripción de reparación.

3. validar datos.

4. cerrar orden y actualizar estado del equipo según reparación.

Alternativas / errores: Si falta fecha, técnico o descripción: impedir cierre. Si continúa sin funcionar, conservar estado compatible y registrar la situación.

Postcondición: Cierre documentado e historial conservado.

CU18. Consultar estado e historial de equipo

Actor: Administrador o guardia.

Precondiciones: Equipo registrado.

Flujo principal:

1. Buscar equipo.

2. mostrar estado, fallas y órdenes de mantenimiento.

Alternativas / errores: Código inexistente: informar sin modificar datos.

Postcondición: Estado e historial consultados.

CU19. Informar cámara sin funcionar

Actor: Sistema, con destinatario administrador.

Precondiciones: Cámara con falla registrada y sin recuperación.

Flujo principal:

1. Contar horas desde la falla.

2. al superar 48 horas, informar al administrador y registrar aviso único.

Alternativas / errores: Menos de 48 horas, cámara reparada o aviso ya emitido: no generar otro.

Postcondición: Aviso de cámara fuera de funcionamiento registrado.

CU20. Autorizar visita o proveedor

Actor: Titular o residente del lote.

Precondiciones: Persona habilitada y lote registrados.

Flujo principal:

1. Ingresar nombre y documento, lote, fechas y franja.

2. elegir puntual o recurrente.

3. validar vigencia.

4. guardar autorización.

Alternativas / errores: Recurrente mayor a 30 días o franja inválida: rechazar. Puntual: un ingreso; recurrente: uno por día según S18.

Postcondición: Autorización disponible para portería.

CU21. Registrar ingreso y egreso

Actor: Guardia de seguridad.

Precondiciones: Visitante identificado por documento.

Flujo principal:

1. Consultar autorización para fecha y hora.

2. verificar vigencia y ausencia de ingreso abierto.

3. registrar ingreso y guardia.

4. al salir registrar egreso.

Alternativas / errores: Sin autorización o fuera de franja: denegar. Si sigue adentro: impedir otro ingreso. Egreso omitido: permitir egreso manual con observación según S19.

Postcondición: Acceso trazable con ingreso y posterior egreso.

CU22. Emitir reportes

Actor: Administrador.

Precondiciones: Datos del sistema disponibles.

Flujo principal:

1. Seleccionar pendientes, morosidad o equipos fuera de servicio.

2. indicar filtros aplicables.

3. recuperar datos.

4. filtrar.

5. calcular capital e intereses para morosidad.

6. mostrar resultado.

Alternativas / errores: Sin coincidencias: mostrar reporte vacío. Filtro inválido: solicitar corrección.

Postcondición: Reporte generado sin modificar registros de negocio.

CU23. Gestionar datos maestros

Actor: Administrador.

Precondiciones: Acceso al módulo de administración.

Flujo principal:

1. Registrar y consultar lotes, personas, vínculos y personal.

2. validar identificadores y superficies.

3. conservar un titular por lote y roles vigentes.

Alternativas / errores: Datos inválidos o identificadores duplicados: rechazar. Cambio de titular excluido por S23.

Postcondición: Datos maestros disponibles para los demás casos.

Trazabilidad

RF1→CU01; RF2→CU02; RF3→CU04/CU07; RF4→CU05; RF5→CU06; RF6→CU03; RF7→CU08; RF8→CU09; RF9→CU10; RF10→CU11; RF11→CU10; RF12→CU12; RF13→CU13; RF14→CU14; RF15→CU15; RF16→CU16; RF17→CU17; RF18→CU18; RF19→CU19; RF20→CU20; RF21-RF22→CU21; RF23→CU22; RF24→CU23.

Nota de cambios del diagrama inicial al final

Propuesta A: Gestión del barrio cerrado Los Aromos.

Esta comparación documenta los diagramas JPG existentes en el repositorio. No afirma que las correcciones pendientes ya se hayan aplicado.

Aspecto	Diseño inicial	Diseño final	Motivo
Coordinación	Gestión concentrada en entidades y GestionDeReclamo	Controladores de Reclamos, Expensa, Equipos, Visitas y Reportes	Separar interacción y coordinación; GRASP Controlador.
Reclamos	GestionDeReclamo y Solicitud separada	GestionReclamos, CambioEstado y Comentarios	Agrupar solicitudes y reclamos conforme S1 y conservar trazabilidad.
Roles de vecinos	Vecino y titular como atributo de Lote	Persona, VinculoLote y RolLote	Permitir distintos roles por persona y lote sin multiplicar subclases.
Expensas	RegistroDeExpensas y Pago con interés	Expensa, Pago, AplicacionPago, AsientoAjuste y CalculadorIntereses	Separar cálculo, imputación y correcciones; preservar pagos.
Parámetros	Valores no centralizados	Configuracion	Concentrar tasa, vencimiento y plazos; variaciones protegidas.
Morosidad	Sin representación específica del reporte	NotificacionMora y FilaMorosidad	Registrar notificaciones y transportar filas del reporte.
Accesos	Visita y RegistroDeVisita	Autorizacion y RegistroAcceso	Separar permiso previo del hecho de ingreso/egreso y permitir recurrencia.
Equipos	Equipo sin abstracción explícita y conexiones de herencia ambiguas	Equipo abstracto, Camara y Luminaria con prioridadOrden	Hacer explícito el polimorfismo para prioridades.
Fallas	RegistroDeFalla	FallaEquipo vinculada a Equipo	Registrar hechos de falla separados del estado actual.
Estados	Estados y tipos parcialmente definidos	Enumeraciones específicas para reclamos, expensas y equipos	Evitar valores ambiguos y distinguir ciclos de vida.

Aclaraciones para la defensa

- Los controladores coordinan; las clases con información y los servicios aplican las reglas correspondientes.

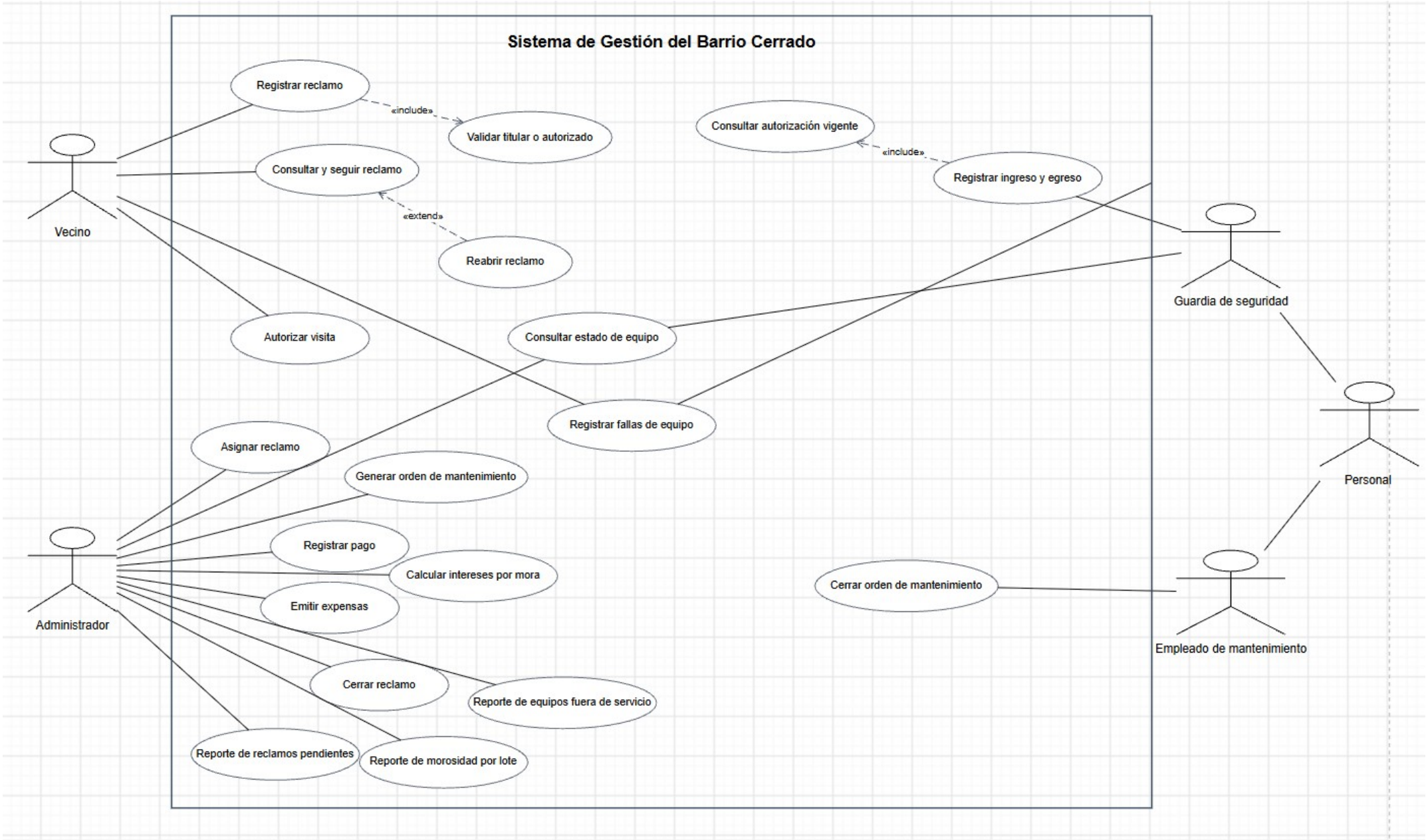
- La composición se usa para partes cuyo ciclo de vida depende del objeto que las contiene, por ejemplo historial y comentarios del reclamo.

- La comparación no demuestra una eliminación de herencia Administrador-Personal: esa herencia no está dibujada en el inicial. La justificación de Liskov debe basarse en relaciones efectivamente presentes.

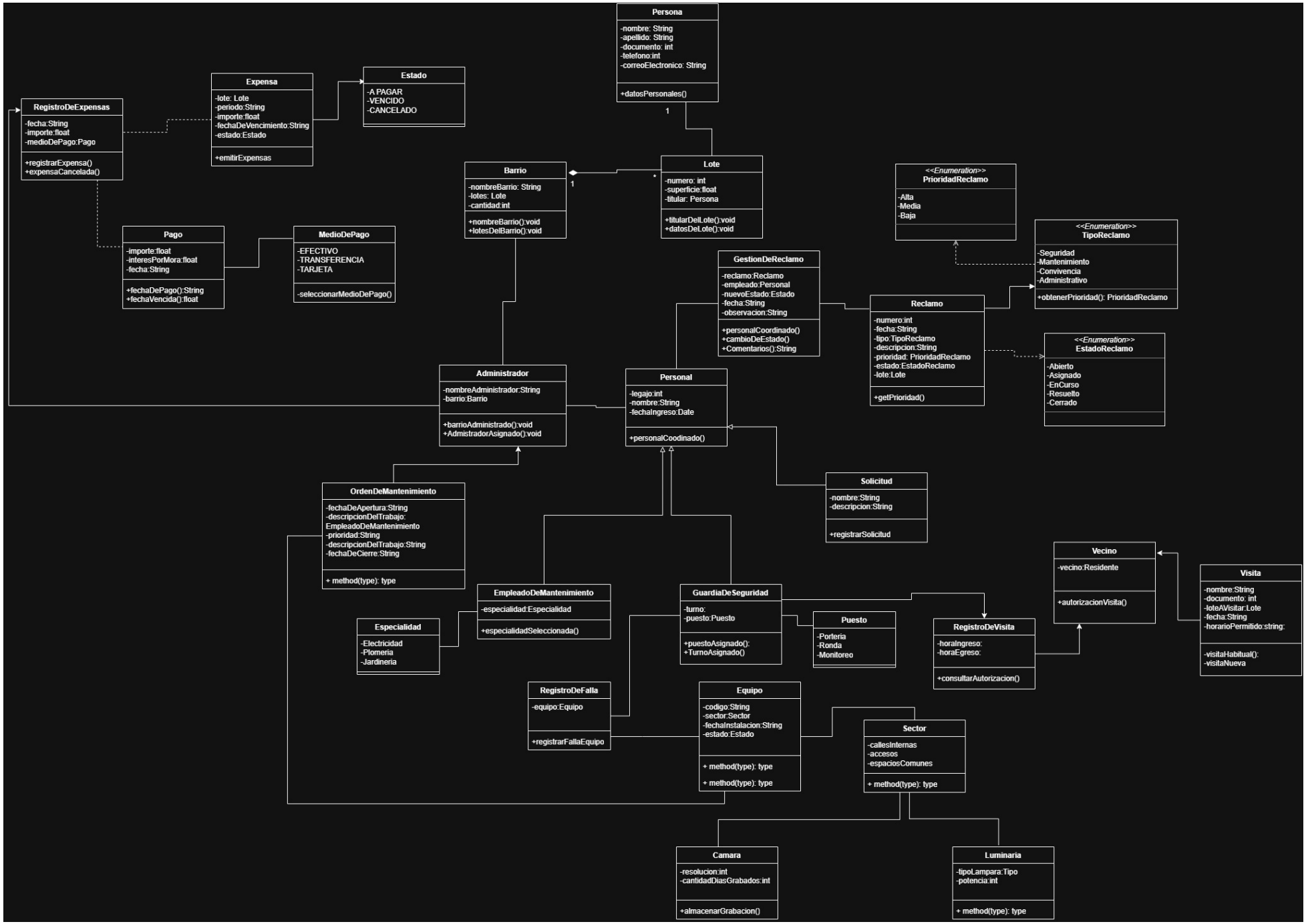
- Persisten diferencias de nombres y firmas entre las imágenes y el informe. En particular, GestorReclamos/GestionReclamos y prioridadOrden con/sin parámetro requieren una decisión común del grupo.

- S23 excluye el cambio de titular, aunque el diagrama final muestra cambiarTitular; se debe resolver esa diferencia antes de cerrar el diseño.

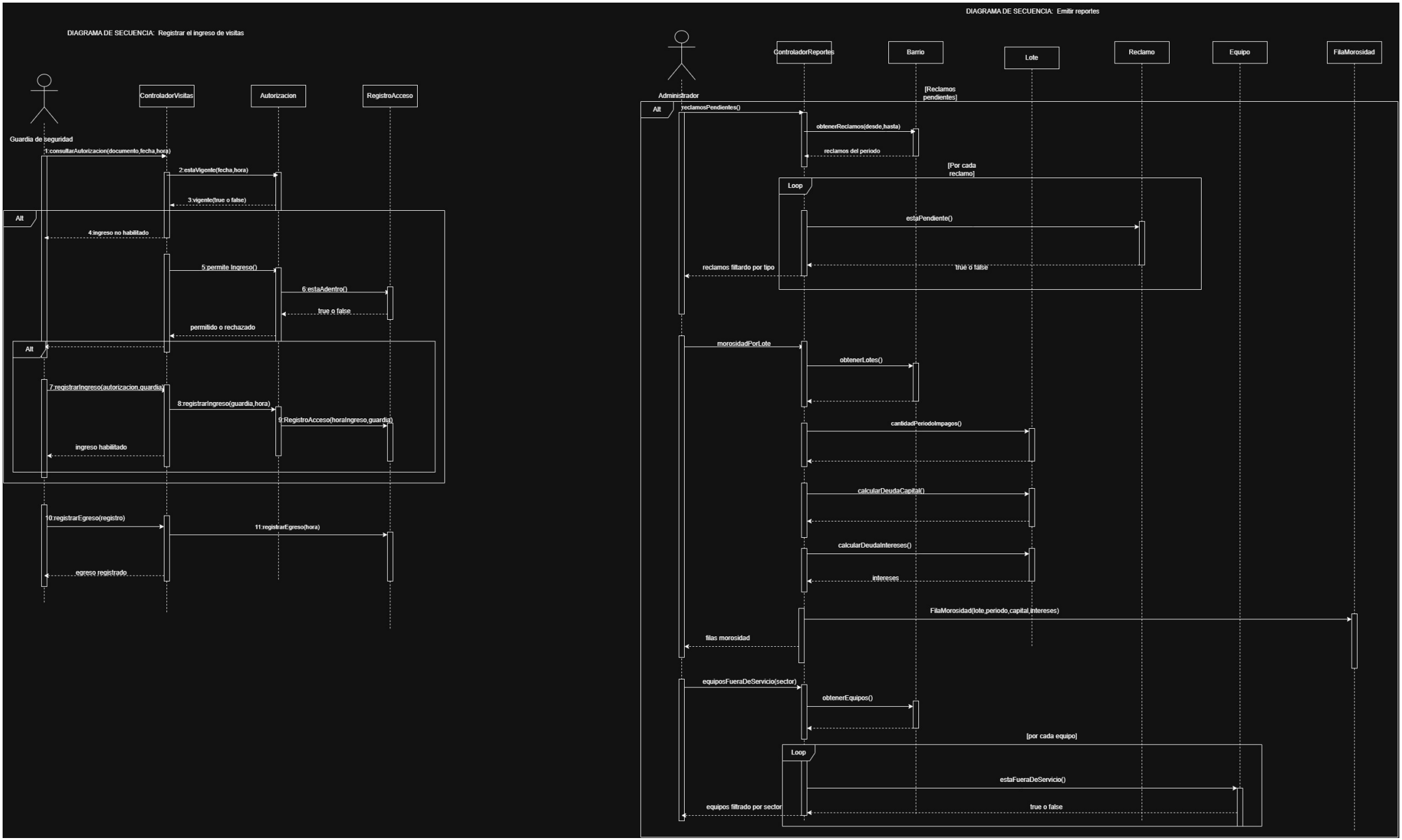
Casos de uso - imagen del repositorio



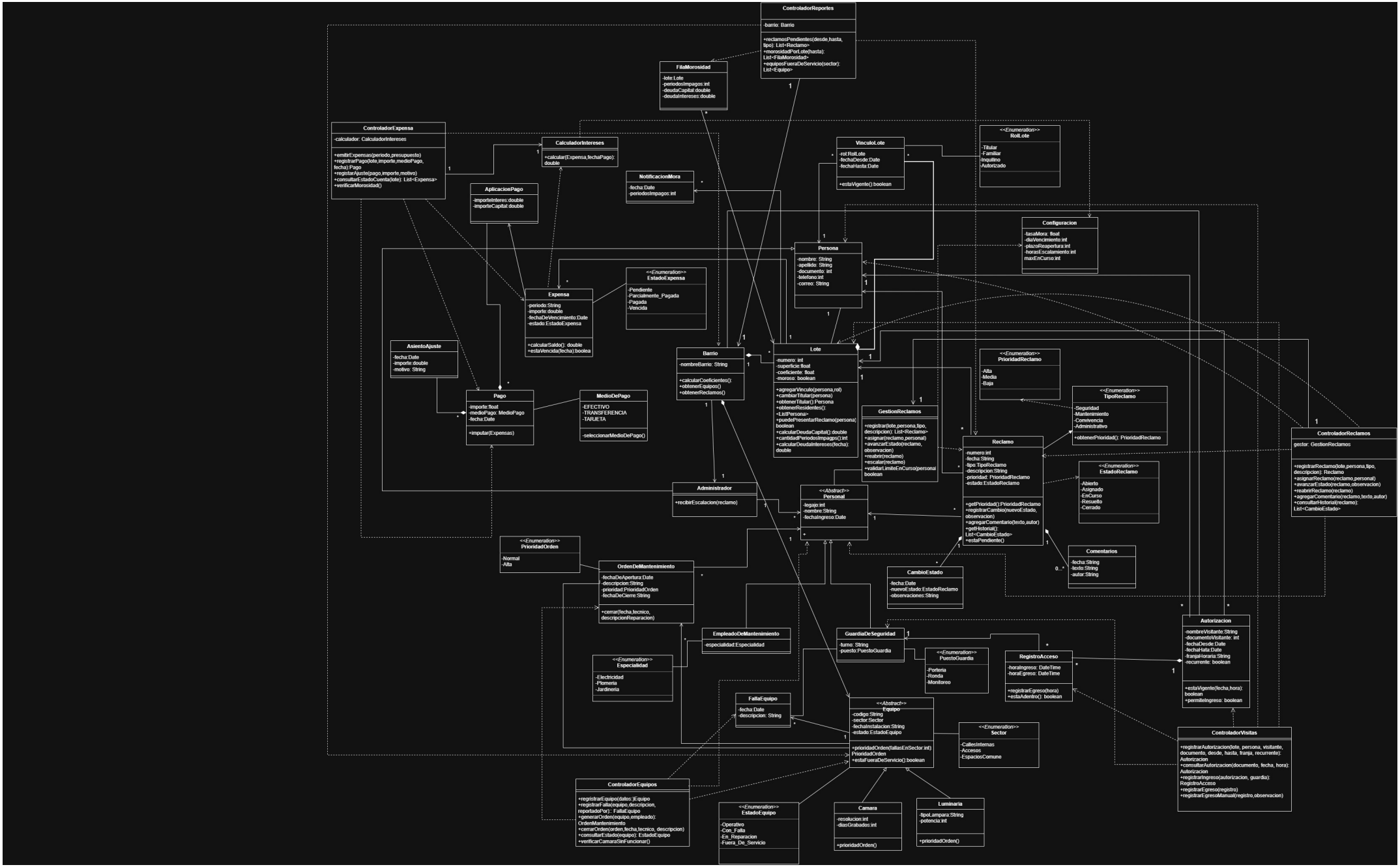
Clases inicial - imagen del repositorio



Secuencias: visitas y reportes - imagen del repositorio



Clases final - imagen del repositorio



Anexo actualizado: coherencia del diseño

Los diagramas fuente editables de esta sección corrigen las inconsistencias entre clases y secuencias detectadas en la primera compilación. Las imágenes JPG originales se mantienen como registro del material previo y no representan esta revisión.

Decisiones aplicadas

• Una firma común para prioridad polimórfica: Equipo.prioridadOrden(fallasEnSector), implementada por Camara y Luminaria; cumple RN15.

• Los nombres se unifican: GestionReclamos, Comentario, ControladorExpensas y OrdenMantenimiento.

• GestionReclamos.registrar devuelve un Reclamo, igual que el controlador.

• Barrio.obtenerLotes permite recorrer los lotes; el reporte solo muestra morosidad con más de tres períodos impagos, como RN12.

• La secuencia de visitas valida autorización y franja, rechaza ingresos duplicados y registra egreso, según RN18–RN20.

• Se excluye cambiarTitular: S23 deja fuera de alcance el cambio; se mantiene un titular por lote, según RN21.

• La herencia Administrador–Personal no aparece en el diagrama inicial; la nota de cambios no afirma que se haya eliminado.

Secuencia de ingreso y egreso

04_secuencia_visitas.puml

```
@startuml
skinparam classAttributeIconSize 0
title Registrar ingreso y egreso de una visita
actor "Guardia de seguridad" as G
participant ControladorVisitas as C
participant Autorizacion as A
participant RegistroAcceso as R
G -> C : consultarAutorizacion(documento, fecha, hora)
C -> A : estaVigente(fecha, hora)
A --> C : vigente
alt autorización vencida o inexistente
 C --> G : denegarIngreso(motivo)
else autorización vigente
 C -> A : permiteIngreso()
 A --> C : permitido
 alt persona ya está dentro
  C --> G : denegarIngreso("egreso pendiente")
 else autorización no habilita esta visita
  C --> G : denegarIngreso("fuera de franja o límite")
 else ingreso habilitado
  G -> C : registrarIngreso(autorizacion, guardia)
  create R
  C -> R : RegistroAcceso(horaIngreso, guardia)
  C --> G : ingreso registrado(registro)
  G -> C : registrarEgreso(registro)
  C -> R : registrarEgreso(hora)
  R --> C : egreso registrado
  C --> G : egreso confirmado
 end
end
@enduml
```

Secuencia de reportes

05_secuencia_reportes.puml

```
@startuml
skinparam classAttributeIconSize 0
title Generar reportes de gestión
actor Administrador as A
participant ControladorReportes as C
participant Barrio as B
participant Lote as L
participant Reclamo as R
participant Equipo as E
participant FilaMorosidad as F
alt reclamos pendientes
 A -> C : reclamosPendientes(desde, hasta, tipo)
 C -> B : obtenerReclamos()
 B --> C : listaReclamos
 loop por cada reclamo
  C -> R : estaPendiente()
  R --> C : pendiente
  opt pendiente y dentro del período y tipo solicitado
   C -> C : agregar al resultado
  end
 end
 C --> A : List<Reclamo>
else morosidad por lote
 A -> C : morosidadPorLote(hasta)
 C -> B : obtenerLotes()
 B --> C : listaLotes
 loop por cada lote
  C -> L : cantidadPeriodosImpagos()
  L --> C : cantidad
  opt cantidad > 3 (RN12)
   C -> L : calcularDeudaCapital()
   L --> C : capital
   C -> L : calcularDeudaIntereses(hasta)
   L --> C : intereses
   C -> F : crear(lote, cantidad, capital, intereses)
  end
 end
 C --> A : List<FilaMorosidad>
else equipos fuera de servicio
 A -> C : equiposFueraDeServicio(sector)
 C -> B : obtenerEquipos()
 B --> C : listaEquipos
 loop por cada equipo
  C -> E : estaFueraDeServicio()
  E --> C : fueraDeServicio
  opt equipo del sector solicitado y fuera de servicio
   C -> C : agregar al resultado
  end
 end
 C --> A : List<Equipo>
end
@enduml
```

Diagrama final de clases

03_clases_final.puml

```
@startuml
skinparam classAttributeIconSize 0
title Clases final - diseño alineado con secuencias y requisitos
class Persona {
-nombre: String
-apellido: String
-documento: int
-telefono: int
-correo: String
}
class Barrio {
-nombreBarrio: String
+calcularCoeficientes()
+obtenerEquipos(): List<Equipo>
+obtenerReclamos(): List<Reclamo>
+obtenerLotes(): List<Lote>
}
class Lote {
-numero: int
-superficie: float
-coeficiente: float
-moroso: boolean
+agregarVinculo(persona, rol)
+obtenerTitular(): Persona
+obtenerResidentes(): List<Persona>
+puedePresentarReclamo(persona): boolean
+calcularDeudaCapital(): double
+cantidadPeriodosImpagos(): int
+calcularDeudaIntereses(hasta: Date): double
}
class VinculoLote {
-rol: RolLote
-fechaDesde: Date
-fechaHasta: Date
+estaVigente(): boolean
}
class Administrador {
+recibirEscalacion(reclamo)
}
abstract class Personal {
-legajo: int
-nombre: String
-fechaIngreso: Date
}
class GuardiaDeSeguridad {
-turno: String
-puesto: PuestoGuardia
}
class EmpleadoDeMantenimiento {
-especialidad: Especialidad
}
class Reclamo {
-numero: int
-fecha: String
-tipo: TipoReclamo
-descripcion: String
-prioridad: PrioridadReclamo
-estado: EstadoReclamo
+getPrioridad(): PrioridadReclamo
+registrarCambio(nuevoEstado, observacion)
+agregarComentario(texto, autor)
+getHistorial(): List<CambioEstado>
+estaPendiente()
}
class GestionReclamos {
+registrar(lote: Lote, persona: Persona, tipo: TipoReclamo, descripcion: String): Reclamo
+asignar(reclamo, personal)
+avanzarEstado(reclamo, observacion)
+reabrir(reclamo)
+escalar(reclamo)
+validarLimiteEnCurso(personal): boolean
}
class CambioEstado {
-fecha: Date
-nuevoEstado: EstadoReclamo
-observaciones: String
}
class Comentario {
-fecha: String
-texto: String
-autor: String
}
class Expensa {
-periodo: String
-importe: double
-fechaDeVencimiento: Date
-estado: EstadoExpensa
+calcularSaldo(): double
+estaVencida(fecha): boolean
```

```
}
class Pago {
-importe: float
-medioPago: MedioDePago
-fecha: Date
+imputar(expensas)
}
class AplicacionPago {
-importeInteres: double
-importeCapital: double
}
class AsientoAjuste {
-fecha: Date
-importe: double
-motivo: String
}
class CalculadorIntereses {
+calcular(expensa, fechaPago): double
}
class Configuracion {
-tasaMora: float
-diaVencimiento: int
-plazoReapertura: int
-horasEscalamiento: int
-maxEnCurso: int
}
class NotificacionMora {
-fecha: Date
-periodosImpagos: int
}
class FilaMorosidad {
-lote: Lote
-periodosImpagos: int
-deudaCapital: double
-deudaIntereses: double
}
abstract class Equipo {
-codigo: String
-sector: Sector
-fechaInstalacion: String
-estado: EstadoEquipo
{abstract} +prioridadOrden(fallasEnSector: int): PrioridadOrden
+estaFueraDeServicio(): boolean
}
class Camara {
-resolucion: int
-diasGrabados: int
+prioridadOrden(fallasEnSector: int): PrioridadOrden
}
class Luminaria {
-tipoLampara: String
-potencia: int
+prioridadOrden(fallasEnSector: int): PrioridadOrden
}
class FallaEquipo {
-fecha: Date
-descripcion: String
}
class OrdenMantenimiento {
-fechaDeApertura: Date
-descripcion: String
-prioridad: PrioridadOrden
-fechaDeCierre: String
+cerrar(fecha, tecnico, descripcionReparacion)
}
class Autorizacion {
-nombreVisitante: String
-documentoVisitante: int
-fechaDesde: Date
-fechaHasta: Date
-franjaHoraria: String
-recurrente: boolean
+estaVigente(fecha, hora): boolean
+permiteIngreso(): boolean
}
class RegistroAcceso {
-horaIngreso: DateTime
-horaEgreso: DateTime
-guardia: GuardiaDeSeguridad
+registrarEgreso(hora)
+estaAdentro(): boolean
}
class ControladorReclamos {
-gestor: GestionReclamos
+registrarReclamo(lote, persona, tipo, descripcion): Reclamo
+asignarReclamo(reclamo, personal)
+avanzarEstado(reclamo, observacion)
+reabrirReclamo(reclamo)
+agregarComentario(reclamo, texto, autor)
+consultarHistorial(reclamo): List<CambioEstado>
}
class ControladorExpensas {
-calculador: CalculadorIntereses
+emitirExpensas(periodo, presupuesto)
+registrarPago(lote, importe, medioPago, fecha): Pago
```

```
+registrarAjuste(pago, importe, motivo)
+consultarEstadoCuenta(lote): List<Expensa>
+verificarMorosidad()
}
class ControladorEquipos {
+registrarEquipo(datos): Equipo
+registrarFalla(equipo, descripcion, reportadoPor): FallaEquipo
+generarOrden(equipo, empleado): OrdenMantenimiento
+cerrarOrden(orden, fecha, tecnico, descripcion)
+consultarEstado(equipo): EstadoEquipo
+verificarCamaraSinFuncionar()
}
class ControladorVisitas {
+registrarAutorizacion(lote, persona, visitante, documento, desde, hasta, franja, recurrente): Autorizacion
+consultarAutorizacion(documento, fecha, hora): Autorizacion
+registrarIngreso(autorizacion, guardia): RegistroAcceso
+registrarEgreso(registro)
+registrarEgresoManual(registro, observacion)
}
class ControladorReportes {
-barrio: Barrio
+reclamosPendientes(desde, hasta, tipo): List<Reclamo>
+morosidadPorLote(hasta): List<FilaMorosidad>
+equiposFueraDeServicio(sector): List<Equipo>
}
enum RolLote {
Titular
Familiar
Inquilino
Autorizado
}
enum EstadoReclamo {
Abierto
Asignado
EnCurso
Resuelto
Cerrado
}
enum PrioridadReclamo {
Alta
Media
Baja
}
enum TipoReclamo {
Seguridad
Mantenimiento
Convivencia
Administrativo
}
enum EstadoExpensa {
Pendiente
Parcialmente_Pagada
Pagada
Vencida
}
enum MedioDePago {
EFECTIVO
TRANSFERENCIA
TARJETA
}
enum PrioridadOrden {
Normal
Alta
}
enum EstadoEquipo {
Operativo
Con_Falla
En_Reparacion
Fuera_De_Servicio
}
enum Sector {
CallesInternas
Accesos
EspaciosComunes
}
enum PuestoGuardia {
Porteria
Ronda
Monitoreo
}
enum Especialidad {
Electricidad
Plomeria
Jardineria
}

Personal <|-- GuardiaDeSeguridad
Personal <|-- EmpleadoDeMantenimiento
Persona <|-- Administrador
Equipo <|-- Camara
Equipo <|-- Luminaria
Barrio "1" *-- "150" Lote
Barrio "1" -- "*" Equipo
Lote "1" *-- "*" VinculoLote
VinculoLote "*" --> "1" Persona
```

```
Lote "1" -- "*" Reclamo
Reclamo "*" --> "1" Persona : solicitante
Reclamo "*" --> "0..1" Personal : responsable
Reclamo "1" *-- "*" CambioEstado
Reclamo "1" *-- "*" Comentario
Lote "1" -- "*" Expensa
Pago "1" *-- "*" AplicacionPago
AplicacionPago "*" --> "1" Expensa
Pago "1" *-- "*" AsientoAjuste
Lote "1" -- "*" NotificacionMora
Equipo "1" -- "*" FallaEquipo
Equipo "1" -- "*" OrdenMantenimiento
OrdenMantenimiento "*" --> "1" Personal
Lote "1" -- "*" Autorizacion
Autorizacion "*" --> "1" Persona : autoriza
Autorizacion "1" *-- "*" RegistroAcceso
RegistroAcceso "*" --> "1" GuardiaDeSeguridad
ControladorReclamos --> GestionReclamos
GestionReclamos ..> Reclamo
GestionReclamos ..> Configuracion
ControladorExpensas --> CalculadorIntereses
CalculadorIntereses ..> Configuracion
ControladorExpensas ..> Pago
ControladorEquipos ..> Equipo
ControladorVisitas ..> Autorizacion
ControladorVisitas ..> RegistroAcceso
ControladorReportes --> Barrio
ControladorReportes ..> FilaMorosidad

@enduml
```